



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА № 7
з дисципліни «Технології розроблення програмного забезпечення»
Тема: «Патерни проектування»

Виконала:

Студентка групи ІА-31

Соколова Поліна

Мета: Вивчити структуру шаблонів «Mediator», «Facade», «Bridge», «Template method» та навчитися застосовувати їх в реалізації програмної системи.

15. E-mail клієнт (singleton, builder, decorator, template method, interpreter, client-server)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Хід роботи

Короткі теоретичні відомості:

Mediator

Використовується для організації взаємодії між об'єктами через один спеціальний координуючий клас. Замість того, щоб об'єкти напрямку посилалися один на одного та складно взаємодіяли, вони надсилають події медіатору, який вирішує, як має реагувати система. Це дозволяє суттєво зменшити кількість зв'язків між компонентами та спрощує модифікацію системи, особливо UI та комплексних взаємодій.

Переваги: зниження зв'язності, централізація логіки взаємодії, легше тестування та розширення.

Недоліки: у великій системі медіатор може перетворитися на «God Object», що містить забагато логіки.

Facade

Надає спрощений інтерфейс до складної підсистеми, приховуючи її внутрішню структуру. Це дозволяє захистити клієнтський код від змін у підсистемі, зменшити кількість залежностей та створити більш зрозумілий API.

Переваги: інкапсуляція складності, спрощення інтерфейсу, покращення читабельності коду.

Недоліки: надмірне використання фасадів може знизити гнучкість доступу до підсистеми.

Bridge

Розділяє абстракцію та реалізацію на дві незалежні ієрархії. Такий підхід є корисним, коли одночасно існує багато варіацій поведінки й багато варіантів реалізації — без Bridge це веде до стрімкого розростання кількості класів.

Переваги: можливість незалежно змінювати абстракцію і реалізацію, зменшення дублювання коду, висока гнучкість.

Недоліки: збільшення кількості класів і проміжних рівнів, складність розуміння на перших етапах.

Template Method

Описує загальну структуру алгоритму в абстрактному класі та дозволяє підкласам перевизначати його окремі кроки. Патерн добре підходить у ситуаціях, коли алгоритми схожі за структурою, але містять відмінні частини.

Переваги: повторне використання коду, можливість легко додавати нові варіації поведінки, чітке розділення загальної та специфічної логіки.

Недоліки: складно підтримувати дуже великий алгоритм; зміни у базовому класі можуть вплинути на всі підкласи; можливе порушення принципу Liskov при невдалій реалізації.

Шаблон «Template method»

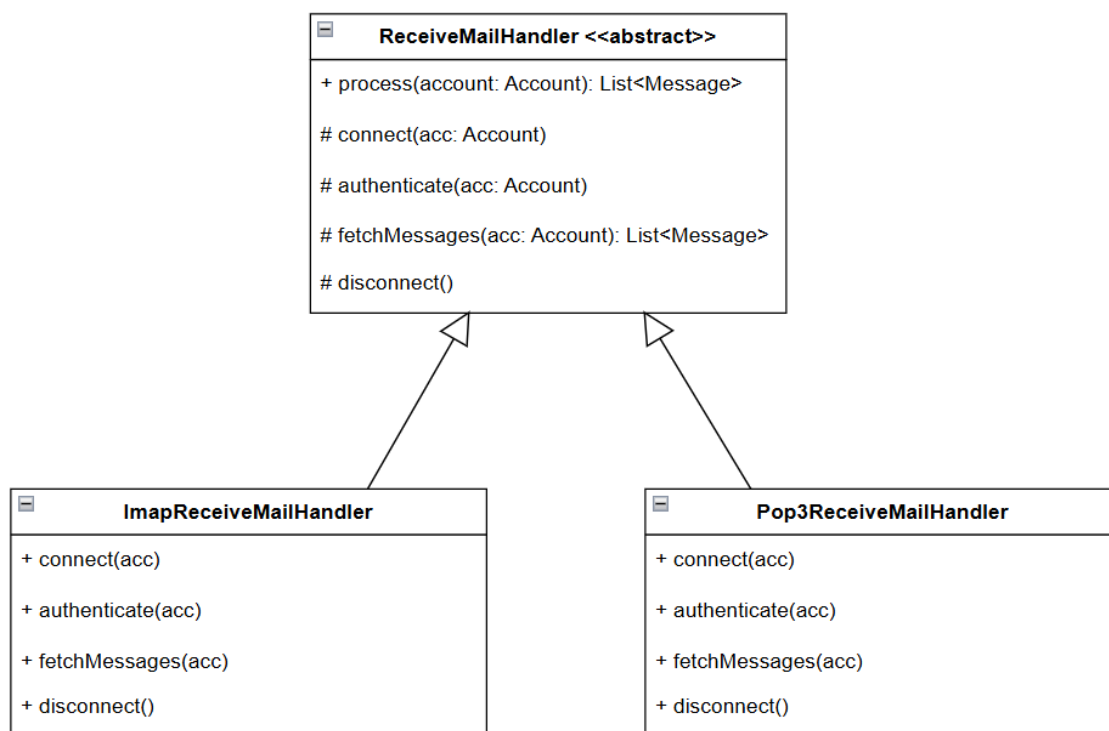


Рис.1 Діаграма класів, яка представляє використання шаблону

У діаграмі класів центральним елементом є абстрактний клас `ReceiveMailHandler`. Він містить шаблонний метод `process()`, який описує послідовність виконання операцій для отримання пошти: встановлення з'єднання, автентифікація, завантаження повідомлень і відключення клієнта. Кожен із цих кроків визначений як абстрактний метод, що означає необхідність реалізації в підкласах.

Підкласами є `ImapReceiveMailHandler` та `Pop3ReceiveMailHandler`, кожен з яких надає власну реалізацію кроків алгоритму. Хоча структура процесу однакова, ІМАР та POP3 працюють по-різному, тому кожен конкретний клас реалізує свої особливості підключення, отримання та закриття сесії. Таким чином, сам алгоритм залишається незмінним, але конкретні дії залежать від протоколу.

Застосування шаблону `Template Method` у системі електронної пошти дозволило уникнути дублювання коду в реалізаціях POP3 та ІМАР. Усі спільні кроки залишились у базовому класі, а специфічні операції перенесені у підкласи. Це

зробило код більш структурованим, зменшило зв'язність і спростило розширення програми — наприклад, додавання нового протоколу тепер зводиться до створення одного класу з потрібними реалізаціями.

```
public abstract class ReceiveMailHandler { 4 usages 2 inheritors

    public final List<Message> process(Account account) { 1 usage
        connect(account);
        authenticate(account);
        List<Message> messages = fetchMessages(account);
        disconnect();
        return messages;
    }

    protected abstract void connect(Account acc); 1 usage 2 implemen
    protected abstract void authenticate(Account acc); 1 usage 2 implemen
    protected abstract List<Message> fetchMessages(Account acc); 1 usage 2 implemen
    protected abstract void disconnect(); 1 usage 2 implementations
}
```

1 }

```
public class Pop3ReceiveMailHandler extends ReceiveMailHandler { 2 usages

    @Override 1 usage
    protected void connect(Account acc) {
        System.out.println("POP3: Connecting to server");
    }

    @Override 1 usage
    protected void authenticate(Account acc) {
        System.out.println("POP3: Authenticating " + acc.getEmail());
    }

    @Override 1 usage
    protected List<Message> fetchMessages(Account acc) {
        System.out.println("POP3: Downloading all messages");
        return new ArrayList<>();
    }

    @Override 1 usage
    protected void disconnect() {
        System.out.println("POP3: Disconnect");
    }
}
```

2 }

```

public class ImapReceiveMailHandler extends ReceiveMailHandler { 2 usages

    @Override 1 usage
    protected void connect(Account acc) {
        System.out.println("IMAP: Connecting");
    }

    @Override 1 usage
    protected void authenticate(Account acc) {
        System.out.println("IMAP: Authenticating " + acc.getEmail());
    }

    @Override 1 usage
    protected List<Message> fetchMessages(Account acc) {
        System.out.println("IMAP: Synchronizing folders");
        return new ArrayList<>();
    }

    @Override 1 usage
    protected void disconnect() {
        System.out.println("IMAP: Disconnect");
    }
}
3 }

public List<Message> receive(int accountId) { no usages new *

    Account acc = accountRepository.getById(accountId);

    ReceiveMailHandler handler = switch (acc.getProtocol()) {
        case POP3 -> new Pop3ReceiveMailHandler();
        case IMAP -> new ImapReceiveMailHandler();
        case SMTP -> throw new IllegalArgumentException(
            "SMTP cannot receive messages.");
    };

    return handler.process(acc);
}
4 }

```

Рис.2 Фрагменти коду по реалізації шаблону
 (1– ReceiveMailHandler, 2 – Pop3ReceiveMailHandler,
 3 – ImapReceiveMailHandler, 4 – MessageService)

Застосування шаблону Template Method дозволило уникнути дублювання коду в реалізаціях POP3 та IMAP. Усі спільні кроки залишились у базовому класі, а специфічні операції перенесені у підкласи. Це зробило код більш структурованим, зменшило зв'язність і спростило можливе майбутнє розширення програми. Додавання нового протоколу тепер зводиться до створення одного класу з потрібними реалізаціями.

Висновок: у даній лабораторній роботі було реалізовано шаблон «Template method», який дозволив винести загальний алгоритм отримання електронних повідомлень у базовий клас і визначити специфічні кроки в підкласах. Це забезпечило повторне використання коду, спростило підтримку системи та зробило її розширення значно простішим.

Відповіді на питання:

1. Яке призначення шаблону «Посередник»?

Шаблон визначає спосіб взаємодії між об'єктами через окремий об'єкт-медіатор, який координує їхню поведінку та зменшує кількість прямих зв'язків між класами.

3. Які класи входять у шаблон «Посередник», та яка між ними взаємодія?

Має клас-медіатор, який містить логіку взаємодії, та класи-колег, які спілкуються між собою лише через медіатор, повідомляючи йому про події.

4. Яке призначення шаблону «Фасад»?

Фасад спрощує доступ до складної підсистеми, надаючи єдиний високорівневий інтерфейс і приховуючи деталі реалізації.

6. Які класи входять у шаблон «Фасад», та яка між ними взаємодія?

Основний клас-фасад викликає методи внутрішніх класів підсистеми й інкапсулює складність їхньої взаємодії. Клієнт працює лише з фасадом.

7. Яке призначення шаблону «Міст»?

Шаблон розділяє абстракцію і реалізацію в окремі ієрархії, дозволяючи змінювати їх незалежно одна від одної.

9. Які класи входять у шаблон «Міст», та яка між ними взаємодія?

Є абстракція (наприклад, Shape) і реалізація (наприклад, DrawAPI). Абстракція містить посилання на реалізацію і делегує їй виконання конкретних операцій.

10. Яке призначення шаблону «Шаблонний метод»?

Цей шаблон визначає загальну структуру алгоритму в базовому класі і дозволяє підкласам перевизначати окремі кроки, не змінюючи сам алгоритм.

12. Які класи входять у шаблон «Шаблонний метод», та яка між ними взаємодія?

Абстрактний клас із шаблонним методом та абстрактними кроками, а також конкретні підкласи, які реалізують ці кроки. Шаблонний метод викликає їх у заданій послідовності.

13. Чим відрізняється шаблон «Шаблонний метод» від «Фабричного методу»?

Template Method задає структуру алгоритму і дозволяє підкласам змінювати частину його поведінки. Factory Method створює об'єкти і дозволяє підкласам визначити, який саме об'єкт буде створено.

14. Яку функціональність додає шаблон «Міст»?

Він забезпечує можливість розвитку великої кількості варіантів абстракцій та реалізацій без експоненційного зростання кількості класів та дозволяє легко розширювати систему в обох напрямках.